PART III. 프롬프트에서 하네스까지

현대 AI 개론

프롬프트 엔지니어링의 한계를 넘어

컨텍스트와 하네스가 만드는 새로운 패러다임

공냥이 2026

목차

09 프롬프트만으로는 끝났다 4

프롬프트를 갈아도 결과가 나아지지 않는 지점이 온다 4

0.25%의 의미: 면접장의 첫 마디 인사 5

대화가 길어지면 AI가 바보가 되는 이유 6

한국어에서 4%는 40%가 된다 6

95%의 시간은 프롬프트 밖에서 쓰인다 7

컨텍스트가 길어지면 성능이 불균일하게 무너진다 7

컨텍스트가 무너지면 벌어지는 일들 8

2025년 6월: 업계가 같은 벽을 보고 같은 이름을 붙인 주 9

그래서 프롬프트는 쓸모없는가 9

프롬프트만으로는 끝났다, 그러나 프롬프트 엔지니어링은 끝나지 않았다 10

10 컨텍스트 엔지니어링: 정보 건축의 시대 11

프롬프트를 넘어서면 정보 건축이 보인다 11

프롬프트 엔지니어링과 무엇이 다른가 12

컨텍스트 윈도우 안에 실제로 들어 있는 것 13

4축 프레임워크: Write, Select, Compress, Isolate 13

압축의 우선순위: 뭘 먼저 줄일 것인가 14

JIT 컨텍스트: RAG의 진화 14

실전 사례: CLAUDE.md가 모든 것을 바꾼다 15

실전: 100개 파일 리팩토링에 4축을 적용하면 15

70 80%의 법칙과 KV-Cache 16

컨텍스트 엔지니어링은 건축이다 16

컨텍스트 엔지니어링의 한계, 그리고 다음 17

11 하네스: AI가 일하게 만드는 소프트웨어 18

세계에서 가장 똑똑한 모델이 가장 무능해지는 순간 18

하네스는 두 가지를 한다 18

CLAUDE.md: 프로젝트의 두뇌를 설계하는 방법 19
도구 정의 설계: ACI가 프롬프트보다 중요하다 20
에이전트 루프: 수집, 행동, 검증, 반복 20
6.7B가 175B를 이기는 이유: 도구의 힘 22
하네스 설정 과정: 새 프로젝트에서 시작하기 22
Before/After: 하네스의 유무가 만드는 차이 23
장기 실행 에이전트: 하루짜리 작업을 맡기려면 23
서브에이전트: 컨텍스트를 분리하는 기술 24
Claude Code의 숫자들: 하네스가 만드는 차이 25
반패턴: 이렇게 하면 망한다 26
에이전틱 루프 설계: 새로운 핵심 스킬 26
하네스 경쟁: Claude Code만 있는 것이 아니다 27
권한과 안전: 돌이킬 수 없는 동작을 어떻게 다루는가 27
하네스의 다음: 외부 세계와의 연결 28

CHAPTER 09

프롬프트만으로는 끝났다

프롬프트 엔지니어링은 AI 활용의 출발점이었다. 그러나 2025년 중반, 실무에서 벽에 부딪힌 사람들이 거의 동시에 같은 결론에 도달했다. 단어 선택만으로는 부족하다. 이 글은 그 벽이 어디에 있고, 왜 생기며, 무엇이 그 너머에 있는지를 다룬다.

프롬프트를 갈아도 결과가 나아지지 않는 지점이 온다

프롬프트 엔지니어링은 LLM에게 지시를 설계해 원하는 행동을 끌어내는 실무 공예다. 모델 가중치를 건드리지 않고 입력 텍스트의 구조, 예시, 맥락, 역할, 출력 형식만으로 행동을 조형한다. GPT-3 논문(Brown 등, 2020)이 보여준 것처럼 충분히 큰 모델은 파인튜닝 없이 프롬프트 안의 몇몇 시연만으로 다양한 과제를 푼다. 이 발견이 현대 프롬프팅의 출발점이었고, 가장 싸고 빠른 성능 레버였다.

이 공예의 도구 상자는 풍부하다. Zero-shot(예시 없이 지시만), Few-shot(3 5개 예시 시연), CoT(사고의 연쇄로 추론 유도), 구조화 출력(XML 태그, JSON 스키마), 메타 프롬프팅(Suzgun, 2024)까지. Anthropic은 프롬프트 설계 이전에 **성공 기준 정의, 경험적 평가 수단, 초기 초안**이라는 세 가지 전제를 요구한다. 방법론은 성숙했다.

그런데 이 공예에는 천장이 있다. 영어로 바꿔보고, 시스템 프롬프트를 구조화하고, Few-shot 예시를 5개, 10개까지 넣어봐도 어느 순간 결과가 더 이상 좋아지지 않는 지점이 온다. 시도한 것들을 정리하면 패턴이 보인다.

시도	효과	한계
영어 프롬프트	일부 작업에서 개선	한국어 결과물이 필요하면 원점
시스템 프롬프트 구조화	처음 3턴은 효과적	이후 모델이 지시를 은근히 무시
Few-shot 예시 추가	품질 향상	컨텍스트 윈도우를 점유해 작업 공간 축소

세 가지 시도 모두 **프롬프트를 더 잘 쓰기**의 범주에 있었고, 전부 컨텍스트 윈도우라는 유한한 공간 안에서의 트레이드오프에 부딪혔다. 프롬프트를 개선하면 프롬프트가 차지하는 토큰이 늘어나고, 그

만큼 다른 정보가 밀려난다. 이것은 제로섬이다. 프롬프트를 최적화하는 행위 자체가 컨텍스트 전체의 균형을 깨뜨리고 있었던 셈이다.

0.25%의 의미: 면접장의 첫 마디 인사

이 숫자를 직시해야 한다. Claude Opus 4.5의 컨텍스트 윈도우가 200K 토큰이었다. 30분 동안 공들여 쓴 프롬프트가 500 토큰이라면, 그것은 전체 윈도우의 0.25%다. 현재 Opus 4.6은 1M 토큰으로 윈도우가 커졌지만 원리는 동일하다.

나머지 99.75%는 무엇으로 채워지는가.

구성요소	비중	누가 넣는가
시스템 프롬프트	5%	하네스가 자동 삽입
프로젝트 설정 (CLAUDE.md 등)	2%	하네스가 자동 로드
도구 정의 (MCP 포함)	3%	하네스가 자동 노출
이전 대화 기록	5%	턴마다 자동 누적
도구 출력 (파일 내용, 검색 결과)	**80%+**	도구 실행 결과
내 프롬프트	**0.25%**	사람이 직접 작성

How Do Coding Agents Spend Your Money?(OpenReview, 2025) 연구에 따르면 에이전틱 코딩에서 도구 응답이 토큰 소비의 대부분을 차지한다. 프롬프트 0.25%와 도구 출력 80%+. 300배 이상의 격차다.

비유하면 이렇다. 면접을 본다고 가정하자. 면접관 앞에 이력서, 포트폴리오, 추천서, 자격증, 이전 회사 평가서가 놓여 있다. 프롬프트는 면접장에 들어서면서 하는 **첫 마디 인사**다. 그 인사를 아무리 완벽하게 해도, 포트폴리오가 엉망이면 면접에서 떨어진다. 프롬프트 엔지니어링은 **인사를 잘 하는 법**을 가르치는 것이다. 중요하다. 하지만 면접의 당락을 결정하는 것은 서류 전체다.

> 프롬프트는 컨텍스트 윈도우의 0.25%에 불과하다. 나머지 99.75%를 무시한 채 0.25%만 다듬는 것은, 면접 인사만 연습하면서 포트폴리오를 방치하는 것과 같다.

대화가 길어지면 AI가 바보가 되는 이유

프롬프트 엔지니어링으로 해결할 수 없는 문제가 있다. 대화가 길어지면 초반 지시가 사라지는 현상이다.

B2B SaaS 마케팅 콘텐츠를 만드는 실전 시나리오를 보자. 첫 턴에 **전문적이되 딱딱하지 않은 톤, CTO/VP Engineering 타겟, 추상적 수식어 금지, 구체적 수치 필수**라고 지시했다. 1 3턴까지는 훌륭하다. 4 5턴부터 금지한 표현이 하나씩 끼어들고, 6턴 이후에는 명시적으로 금지한 표현이 버젓이 나온다.

왜 이런 일이 벌어지는가. 트랜스포머의 어텐션 메커니즘이 U자형 분포를 보이기 때문이다. 앞부분(시스템 프롬프트 근처)과 뒷부분(최근 대화)에는 높은 주의를 기울이지만, 가운데(초반 대화 기록)에는 낮은 주의를 기울인다. 이것이 **Lost-in-Middle** 현상이다.

2026년의 모델들은 컨텍스트 윈도우가 1M 토큰까지 늘어났고 어텐션 메커니즘도 개선됐다. 예전처럼 7턴이 지나면 무조건 망하지는 않는다. 하지만 근본 원리는 변하지 않았다. 대화가 충분히 길어지면 초반 지시가 가운데로 밀려나면서 영향력이 줄어든다. 윈도우가 커졌으니 발생 시점이 늦춰졌을 뿐이다.

이 문제에 대한 나쁜 대응과 좋은 대응이 있다. 나쁜 대응은 **아까 한 거 잊지 마**라고 다시 말하는 것이다. 프롬프트로 프롬프트 문제를 해결하려는 시도인 셈이다. 일시적으로 듣지만 몇 턴 뒤에 또 반복된다. 근본 해결이 아니라 땜빵이다.

좋은 대응은 세 가지다. 첫째, 시스템 프롬프트에 핵심 규칙을 넣는다. 시스템 프롬프트는 컨텍스트의 맨 앞에 위치하므로 어텐션이 항상 높다. 둘째, 세션을 짧게 끊고 핵심 규칙과 함께 새 세션을 시작한다. 컨텍스트를 초기화하는 것이다. 셋째, CLAUDE.md 같은 프로젝트 설정 파일을 활용한다. 매 세션 시작 시 자동으로 로드되므로 밀릴 수가 없다.

프롬프트는 대화 기록 안에 들어가서 뒤로 밀린다. CLAUDE.md는 시스템 수준에서 매 턴마다 로드된다. 이것은 프롬프트를 잘 쓰는 기술이 아니라 **컨텍스트를 관리하는 기술**이다. 세션을 짧게 쓰고 바로바로 새 대화로 넘어가라는 조언도 프롬프트 팁이 아니라 컨텍스트 관리 원칙인 셈이다.

한국어에서 4%는 40%가 된다

벤치마크에서 한국어 성능이 영어와 4% 차이라는 결과가 나와도, 실전에서 한국어로 복잡한 작업을 시키면 체감 차이는 40%에 이른다. 이것을 **한국어가 고맥락 언어라서**라고만 설명하면 절반만 맞는 이야기가 된다.

한국어는 Edward Hall의 분류에서 고맥락 언어에 속한다. 명시적 정보가 약 40%에 불과하고, 나머지 60%는 상황, 관계, 문화적 뉘앙스에 담겨 있다. **생각 좀 해볼게요**가 거절이라는 것을 AI가 모르는 이유도 여기에 있다. 그러나 진짜 원인은 한국어 자체가 아니라 **컨텍스트**다.

고객 응대 메일을 AI로 작성하는 실전 사례를 보자. **고객에게 사과 메일을 써줘, 정중하게**라고만 프롬프트를 주면, AI는 문법적으로 완벽하지만 한국인이 읽으면 1초 만에 봇이 쓴 것임을 알아채는 메일을 생성한다. 한국어 비즈니스 사과 메일에는 **상황 인정, 원인 설명, 대책 제시, 보상 언급, 재발 방지 약속**이라는 암묵적 순서가 있기 때문이다.

반면 발신자의 직급과 성별, 수신자와의 거래 기간, 불량 수량과 현재 상태, 관계 유지의 중요도를 컨텍스트로 제공하면 결과가 완전히 달라진다. AI가 **상황 인정(불량 23개를 확인했다는 것), 원인 분석(품질 관리 프로세스의 구체적 문제), 대책(교체품 발송 일정), 재발 방지(검수 강화 계획)**의 순서로 메일을 구성한다. 이것은 프롬프트를 잘 쓴 것이 아니라, **이게 상황이다**라는 정보를 윈도우에 넣은 것이다.

프롬프트는 **이렇게 해줘**라는 지시이고, 컨텍스트는 **이게 상황이다**라는 정보다. 지시만 있고 상황이 없으면, 아무리 지시를 잘 해도 상황에 맞는 결과가 나올 수 없다. **한국어의 고맥락 특성을 고려해서 응답해줘**라고 프롬프트에 써봐야, 그 맥락 정보 자체가 윈도우에 없으면 모델이 참조할 것이 없다.

95%의 시간은 프롬프트 밖에서 쓰인다

지금 실무에서 AI로 작업할 때 프롬프트를 다듬는 데 쓰는 시간은 전체의 5% 이하다. 나머지 95%는 다른 곳에 쓰인다. **이 맥락에서 어떤 파일을 읽게 할까, 이 정보를 어떤 순서로 제공할까, 이전 출력에서 뭘 남기고 뭘 버릴까, 이 작업은 서브에이전트에게 격리해서 시킬까.** 이것을 **프롬프트 엔지니어링**이라고 부르는 것은 무리가 있다. 이것은 정보 건축이다.

엑셀에 비유하면 이해가 빠르다. 프롬프트를 다듬는 것은 엑셀 함수를 외우는 것이다. 컨텍스트를 관리하는 것은 데이터 구조를 설계하는 것이다. 함수를 아무리 잘 써도 데이터가 엉망이면 결과가 엉망이다. VLOOKUP을 완벽하게 쓸 수 있어도, 원본 데이터의 열 구조가 잘못돼 있으면 아무 소용이 없다.

Harrison Chase(LangChain)가 이 현상을 정확히 짚었다. **모델이 좋아질수록, 실패는 빈약한 phrasing에서 부적절한 컨텍스트로 수렴한다.** GPT-3 시절에는 프롬프트를 **think step by step**이라고 바꾸는 것만으로 수학 성능이 수십 퍼센트 뛰었다. 2026년 Opus 4.6 시대에는 그런 차이가 미미하다. 모델이 충분히 똑똑해서 웬만한 문구는 다 이해하기 때문이다. 진짜 차이를 만드는 것은 어떤 정보가 윈도우에 들어 있느냐다.

컨텍스트가 길어지면 성능이 불균일하게 무너진다

직관에 반하는 연구 결과가 있다. Chroma Research(Kelly Hong 외, 2025)가 18개 SOTA 모델을 벤치마크한 결과, 컨텍스트가 논리적으로 일관될수록 오히려 성능이 나빠졌다.

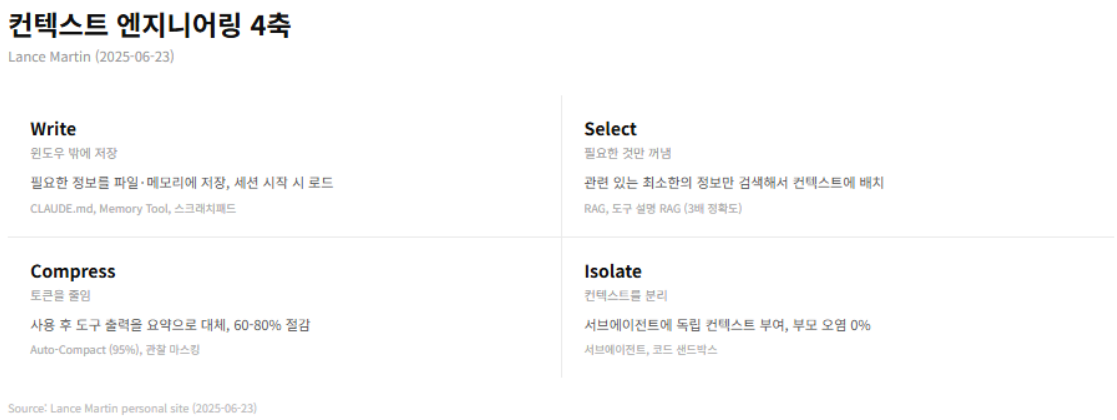

그림 1: 컨텍스트 실패의 4가지 모드. 오염, 산만, 혼란, 충돌이 각각 다른 원인과 대응을 갖는다.

무작위 텍스트 속에 바늘을 숨기면 잘 찾는다. **Needle in Haystack** 벤치마크에서 높은 점수가 나오는 이유다. 하지만 현실 문서는 논리적으로 구조화돼 있고, 모든 내용이 관련 있어 보인다. 이런 문서에서는 모델이 핵심을 놓친다. 200K 토큰 모델이 50K 토큰 지점에서 이미 불균일한 성능 붕괴를 보였다. 윈도우의 25%만 채웠는데도.

불균일이 핵심이다. 균일하게 나빠지면 예측이라도 가능하지만, 어떤 질문에는 완벽히 답하고 바로 다음 질문에는 완전히 실패하는 패턴이다. 어떤 질문이 실패할지 미리 알 수 없으므로 디버깅이 불가능하다.

Lance Martin은 이 현상을 4가지 실패 모드로 분류했다.

실패 모드	원인	대응
Context Poisoning(오염)	환각이 컨텍스트에 들어가 자기강화 루프 형성	검증 게이트, 메모리 쓰기 승인
Context Distraction(산만)	무관한 정보가 핵심 신호를 희석	Select 축 강화, 요약 기반 검색
Context Confusion(혼란)	도구와 규칙이 너무 많아 모델이 어느 규칙을 따를지 결정 불가	도구/규칙 축소, Isolate
Context Clash(충돌)	초반 지시와 후반 지시가 정면 충돌	주기적 재지시, 요약 재주입

이 네 가지를 프롬프트 엔지니어링에서는 논의조차 하지 않았다. 프롬프트 엔지니어링은 **한 턴의 입력**만 다루기 때문이다. 컨텍스트 전체를 조망하는 시야가 없었던 셈이다.

컨텍스트가 무너지면 벌어지는 일들

컨텍스트 문제를 인식하면, 그동안 **AI가 멍청해서**라고 생각했던 현상들이 전부 설명된다.

첫째, **AI가 갑자기 바보가 됐다**는 현상이다. 대화 10턴째에 앞서 합의한 내용을 뒤집는다. 한국어로 쓰기로 했는데 갑자기 영어로 쓴다. 많은 사람이 이때 **아까 한국어로 쓰라고 했잖아**라고 프롬프트를 강하게 준다. 진짜 원인은 초반 지시가 10턴이 지나면서 컨텍스트의 가운데로 밀렸고, 최근 턴에서 영어 레퍼런스를 참조하면서 모델이 최신 토큰의 언어를 따라간 것이다. Context Clash다.

둘째, **같은 프롬프트인데 어제는 됐고 오늘은 안 되는** 현상이다. 프롬프트는 동일하다. 하지만 어제는 새 세션의 첫 턴이었고, 오늘은 이미 5턴 대화가 쌓인 세션이다. 프롬프트가 전체의 0.25%라면, 나머지 99.75%가 다른데 같은 결과를 기대하는 것이 오히려 이상하다.

셋째, **30개 도구를 연결했는데 오히려 느려지는** 현상이다. MCP로 GitHub, Slack, Notion, DB, 검색엔진을 전부 연결하면 30개 도구 정의가 컨텍스트에 수천 토큰을 차지한다. 모델이 30개 중에서 골라야 하므로 선택지가 많아질수록 정확도가 떨어진다. 버그를 고치라고 했는데 Slack 채널부터 검색하고, Notion 문서를 뒤지고, 세 번의 불필요한 도구 호출 끝에야 GitHub 이슈에서 관련 정보를 찾는 식이다. 이것이 Context Confusion이다.

이 현상들 전부, 프롬프트를 고치느라 시간을 쓰면 영원히 해결되지 않는다. 컨텍스트를 봐야 한다.

2025년 6월: 업계가 같은 벽을 보고 같은 이름을 붙인 주

제가 혼자 느끼고 있던 한계를 업계가 거의 동시에 인정한 주가 있었다. 2025년 6월 셋째 주다.

6월 19일, Shopify CEO Tobi Lutke가 트윗을 올렸다. **태스크가 LLM에 의해 그럴듯하게 풀릴 수 있도록 필요한 모든 컨텍스트를 제공하는 예술**. 이것을 **Context Engineering**이라 불렀다. 연구자가 아니라 매일 AI를 비즈니스에 쓰는 CEO가 먼저 이름을 붙인 것이다.

6월 23일, LangChain의 Harrison Chase가 동조했다. **LLM 앱이 단일 프롬프트에서 복잡한 동적 에이전틱 시스템으로 진화하고 있다.** 6월 25일, Andrej Karpathy가 결정타를 날렸다.

> **Context engineering is the delicate art and science of filling the context window with just the right information for the next step.** 다 넣는 게 아니라 딱 맞는 것만. 전체가 아니라 바로 다음 한 단계를 위해.

6월 27일, Simon Willison이 마무리했다. **Context engineering**은 **prompt engineering의 후속 용어**라고 명시한 것이다. 일주일 만에 합의가 형성됐다.

이것은 트렌드가 아니었다. 실무에서 벽에 부딪힌 사람들이 거의 동시에 같은 결론에 도달한 것이다. CEO, 프레임워크 제작자, 연구자, 비평가가 서로 다른 위치에서 같은 벽을 보고 같은 이름을 붙였다.

그래서 프롬프트는 쓸모없는가

아니다. 프롬프트는 여전히 기초 체력이다. 단발성 대화에서는 여전히 핵심이고, CoT(사고의 연쇄), Few-shot, 구조화 출력은 2026년에도 작동한다. Mollick이 말한 것처럼 **참을성 있는 신입 동료에게 맥락과 예시와 단계를 설명하듯** 프롬프트를 쓰는 원칙은 변하지 않았다. Anthropic의 골든 룰도 유효하다. **맥락 없는 동료에게 프롬프트를 보여 따라해 보게 해라. 그가 헷갈리면 모델도 헷갈린다.**

프롬프트 기법 자체도 계속 발전하고 있다. Wei(2022)의 Few-shot CoT, Wang의 Self-Consistency, Yao의 Tree of Thoughts, Suzgun(2024)의 Meta-Prompting까지. 특히 Meta-Prompting은 단일 모델이 여러 전문가 인스턴스를 오케스트레이션하는 방식으로 GPT-4 기준 표준 프롬프팅 대비 17% 개선을 달성했다. 기법은 살아 있다.

하지만 3시간짜리 코딩 세션, 100개 파일 리팩토링, 일주일짜리 프로젝트에서는 프롬프트의 기여도가 5% 이하로 떨어진다. 나머지 95%는 **어떤 파일을 읽게 할까, 어떤 순서로 정보를 제공할까, 이전 출력에서 뭘 남기고 뭘 버릴까**를 결정하는 데 쓰인다. 이것을 프롬프트 엔지니어링이라 부르기는 어렵다. 이것은 **정보 건축**이다.

Harrison Chase의 관찰이 정확하다. 모델이 좋아질수록 실패는 **빈약한 phrasing**에서 **부적절한 컨텍스트**로 수렴한다. GPT-3 시절에는 프롬프트가 진짜 차이를 만들었다. 2026년 Opus 4.6 시대에는 모델이 충분히 똑똑해서 웬만한 문구는 다 이해한다. 진짜 차이를 만드는 것은 어떤 정보가 컨텍스트에 들어 있느냐다.

프롬프트만으로는 끝났다, 그러나 프롬프트 엔지니어링은 끝나지 않았다

이 글의 제목이 **프롬프트만으로는 끝났다**인 이유를 정리해야 한다. 프롬프트 엔지니어링을 부정하는 것이 아니다. 프롬프트 엔지니어링을 더 큰 그림 안에 배치하는 것이다.

프롬프트만 다듬는 사람은 5%의 레버리지에 시간을 쏟고 있는 셈이다. 95%의 레버리지가 컨텍스트 관리에 있다는 것을 모른 채로. 이것은 엑셀 함수를 외우면서 데이터 구조를 설계하지 않는 것과 비슷하다. 함수를 아무리 잘 써도 데이터가 엉망이면 결과가 엉망이다.

프롬프트 엔지니어링의 안티패턴도 이 맥락에서 재해석된다. **10가지 마법 프롬프트**를 맹신하는 것, **너는 세계 최고 전문가다**라는 한 줄이 마법처럼 성능을 올린다고 믿는 것, 모든 문제를 프롬프팅으로 풀려 하는 것. 이것들은 프롬프트가 전부라는 착각에서 비롯된다. 성공 기준과 평가가 없으면 무한 튜닝만 반복될 뿐이다.

그 더 큰 그림, 컨텍스트를 설계하는 기술이 무엇인지를 다음 장에서 다룬다.

Sources: Brown et al. (GPT-3) 2020; Mollick 2023; Chroma Research (Kelly Hong 외) 2025; Tobi Lutke 2025-06-19; Harrison Chase 2025-06-23; Andrej Karpathy 2025-06-25; Simon Willison 2025-06-27; OpenReview 2025; Edward Hall — High/Low-Context Cultures

CHAPTER 10

컨텍스트 엔지니어링: 정보 건축의 시대

프롬프트만으로는 끝났다면, 대체 뭘 해야 하는가. 답은 컨텍스트 윈도우 전체를 설계하는 것이다. 어떤 정보를, 어떤 형태로, 어떤 순서로, 얼마나 넣을 것인가. 이 설계 행위에 2025년 6월, 이름이 붙었다. 컨텍스트 엔지니어링.

프롬프트를 넘어서면 정보 건축이 보인다

앞 장에서 프롬프트가 컨텍스트 윈도우의 0.25%에 불과하다는 사실을 확인했다. 나머지 99.75%를 채우는 것이 진짜 작업이라면, 그 작업에는 어떤 원칙이 필요한가.

컨텍스트 엔지니어링(Context Engineering)은 LLM 추론 시점에 모델이 보는 토큰 집합 전체를 큐레이션하고 유지하는 전략의 집합이다. 시스템 프롬프트, 도구 정의, MCP, 검색 결과, 메모리, 메시지 이력, 상태를 포함한다. Andrej Karpathy가 2025년 6월 25일에 내린 정의가 이 분야의 출발점이 됐다. **다음 단계를 위해 컨텍스트 윈도우를 딱 알맞은 정보로 채우는 섬세한 예술이자 과학**이라는 것이다.

이 정의에서 빠뜨리면 안 되는 단어가 두 개 있다. **딱 맞는 정보만(just the right)**과 **바로 다음 한 단계를 위해(for the next step)**다.

첫째, 많이 넣는다고 좋지 않다. Chroma Research(Kelly Hong 외, 2025)가 18개 SOTA 모델을 벤치마크한 결과, 컨텍스트가 논리적으로 일관될수록 오히려 성능이 나빠졌다. 200K 토큰 모델이 50K 토큰 지점에서 이미 불균일한 성능 붕괴를 보였다. 윈도우의 25%만 채웠는데도 무너진 셈이다.

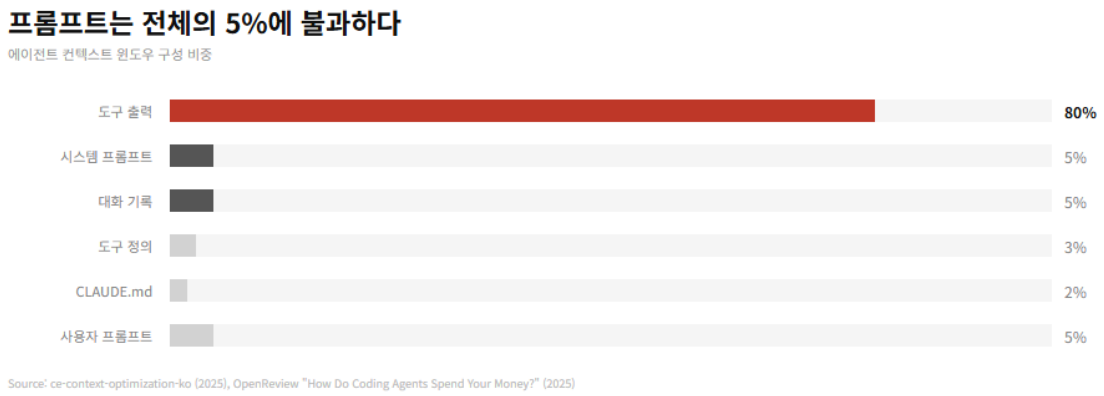

그림 2: 컨텍스트 윈도우의 구성. 시스템 프롬프트, 도구 정의, 검색 문서, 대화 기록, 도구 출력의 5개 영역으로 나뉘며 도구 출력이 80%+를 차지한다.

둘째, 컨텍스트는 정적이 아니라 동적이다. 프롬프트는 대화 시작 전에 한 번 쓰고 끝이다. 컨텍스트 엔지니어링에서는 매 스텝마다 윈도우 내용이 바뀐다. 버그를 찾을 때는 에러 로그와 관련 파일을 로드하고, 원인을 찾았으면 에러 로그를 빼고 수정 대상 파일만 유지하고, 테스트를 돌릴 때는 수정 파일과 테스트 파일만 남긴다. 이전 스텝에서 필요했던 정보가 다음 스텝에서는 노이즈가 되기 때문이다.

프롬프트 엔지니어링과 무엇이 다른가

Harrison Chase(LangChain)가 두 개념의 관계를 정리했다. 프롬프트 엔지니어링은 한 문장의 문구를 다듬는 공예이고, 컨텍스트 엔지니어링은 윈도우에 무엇이 어떤 형식으로 언제 들어가는지의 정보 건축(information architecture)이다. 관계는 포함이다. 프롬프트 엔지니어링은 컨텍스트 엔지니어링의 부분 집합이다.

건물을 짓는 것에 비유하면 이해가 빠르다. 프롬프트를 쓰는 것은 벽에 페인트칠을 하는 것이다. 컨텍스트를 설계하는 것은 건물의 골조를 세우는 것이다. 구조 설계가 4축 프레임워크이고, 자재 선정이 어떤 소스를 넣을 것인가이며, 동선 계획이 정보 배치 순서(캐시 효율)이고, 하중 계산이 토큰 예산 관리(70 80%)이며, 내진 설계가 실패 모드 방어다. 페인트칠도 중요하지만, 골조가 부실하면 건물이 무너진다.

항목	프롬프트 엔지니어링	컨텍스트 엔지니어링
대상	한 턴의 입력 문구	윈도우 전체의 토큰 구성
시점	대화 시작 전 1회	매 스텝마다 동적 갱신
범위	단어 선택, 예시, 역할	시스템 프롬프트 + 도구 + 메모리 + 이력 + 도구 출력
비유	벽 페인트칠	건물 골조
관계	부분 집합	상위 집합

컨텍스트 윈도우 안에 실제로 들어 있는 것

많은 사람이 컨텍스트를 자신이 쓴 메시지로만 생각한다. 실제로 컨텍스트 윈도우를 열어보면 다섯 가지가 들어 있다. 시스템 프롬프트, 도구 정의, 검색된 문서, 대화 기록, 도구 출력이다.

구성요소	역할	문제
시스템 프롬프트	모델의 성격과 규칙 정의	건드리면 기본 행동이 바뀜
도구 정의	사용 가능한 도구 목록	MCP가 많으면 수천 토큰 점유
검색된 문서	RAG/JIT로 가져온 지식	관련 없는 것도 섞임
대화 기록	이전 턴들의 누적	길어지면 Lost-in-Middle 발생
도구 출력	파일 읽기, 검색 결과 등	**전체의 80%+ 차지**

마지막 항목이 결정적이다. 도구 출력이 전체의 80%+를 차지한다. AI에게 파일 10개를 읽게 시키면 그 10개 파일의 전체 내용이 컨텍스트에 들어간다. 파일 하나가 1,000줄이면 10,000줄, 토큰으로 수만 토큰이다. 실제로 필요한 부분은 100줄 정도일 것이다. 나머지 9,900줄은 노이즈다. 이 노이즈가 쌓이면 Chroma가 발견한 불균일한 성능 붕괴가 발생한다.

이것이 프롬프트 엔지니어링으로 해결 불가능한 이유다. 프롬프트가 아무리 좋아도, 컨텍스트 윈도우에 5만 토큰 이상의 무관한 도구 출력이 쌓여 있으면 모델의 행동이 예측 불가능해진다. 문제는 프롬프트가 아니라 프롬프트를 둘러싼 나머지 99.75%에 있다.

4축 프레임워크: Write, Select, Compress, Isolate

Lance Martin이 2025년 6월 23일에 제시한 4축이 컨텍스트 엔지니어링의 표준 프레임워크가 됐다. LLM을 운영체제에, 윈도우를 RAM에 비유한 분류다.

> 4축은 **Write**(윈도우 밖에 저장), **Select**(필요한 조각만 꺼내옴), **Compress**(토큰 축소), **Isolate**(컨텍스트 분리)다. 이 네 축을 조합해 유한한 윈도우 안에서 최대 신호 대 잡음비를 달성한다.

Write 축은 컨텍스트를 윈도우 밖에 저장하는 것이다. 컨텍스트 윈도우는 유한하므로 필요 없는 것은 밖으로 빼야 한다. Claude Code의 CLAUDE.md가 대표적 구현이다. 프로젝트 루트에 규칙 파일을 저장하면 매 세션 시작 시 자동으로 로드된다. 윈도우를 상시 점유하지 않으면서도 필요할 때 항상 참조할 수 있다. 2025년 9월 Anthropic이 이 원칙을 플랫폼에 적용한 Context Editing과 Memory Tool은 장기 에이전트 성능을 39% 향상시켰다.

Select 축은 필요한 조각만 꺼내오는 것이다. 프로젝트의 모든 파일을 읽는 대신, grep으로 특정 함수만 추출해 100줄만 로드한다. 도구 설명 RAG라는 기법도 있다. MCP로 도구가 30개 노출됐을 때 현재 작업에 관련된 도구만 검색해서 노출하면 도구 선택 정확도가 3배 올라간다.

Compress 축은 같은 의미를 더 적은 토큰으로 전달하는 것이다. Claude Code는 컨텍스트가 95% 임계값을 넘으면 자동으로 이전 대화를 요약한다. 관찰 마스킹(Observation Masking)이라는 기법은 도구 출력을 사용한 후 상세 내용을 압축 참조로 대체한다. 1,200줄짜리 파일 내용이 한 줄 요약으로 바뀌면 60 80% 토큰 절감이 가능하다. 품질 손실은 5% 미만이다.

Isolate 축은 하나의 거대한 컨텍스트 대신 여러 개의 격리된 컨텍스트로 나누는 것이다. 메인 에이전트가 전체를 관리하고, 서브에이전트가 각자 독립된 컨텍스트에서 탐색 작업을 수행한 뒤 1,000 2,000 토큰의 응축 요약만 메인으로 돌려보낸다.

여기서 중요한 논쟁이 있다. Cognition(Devin 팀)은 멀티에이전트를 쓰지 말라고 권고하고, Anthropic은 병렬 서브에이전트로 리서치 성능이 90% 향상됐다고 보고한다. 이 둘은 모순이 아니다. Cognition이 말하는 것은 컨텍스트를 **공유하려는** 멀티에이전트의 실패이고, Anthropic이 말하는 것은 컨텍스트를 **격리하는** 서브에이전트의 성공이다. 분리해야 하는 것은 역할이 아니라 컨텍스트다. **코드 작성자**와 **리뷰어**로 역할을 나누면 실패한다. 둘이 같은 컨텍스트를 공유해야 하기 때문이다. **메인 작업**과 **독립 탐색 작업**으로 컨텍스트를 나누면 성공한다.

압축의 우선순위: 뭘 먼저 줄일 것인가

4축 중 Compress를 쓸 때 무엇을 먼저 압축해야 하는지 순서가 있다. 도구 출력이 80%+를 차지하므로 가장 먼저 압축한다. 두 번째는 오래된 턴이다. 세 번째는 검색된 문서다. 시스템 프롬프트는 절대 압축하지 않는다. 시스템 프롬프트를 압축하면 모델의 기본 행동이 바뀌기 때문이다.

도구 출력에서 절감하는 효과가 가장 크다. 파일을 읽은 후에 핵심만 남기고 나머지를 요약하면 60 80% 절감이 가능하다. KVTC(2025)라는 기법은 신호 처리 기법으로 KV-Cache를 20 40배 손실 압축한다. 프레임워크 수준에서 작동하는 기법이라 사용자가 직접 쓸 일은 없지만, 앞으로 컨텍스트 비용이 구조적으로 낮아질 것이라는 신호다.

JIT 컨텍스트: RAG의 진화

전통적 RAG는 쿼리 직전에 임베딩 검색으로 문서를 미리 모아 프롬프트에 붙이는 방식이다. JIT(Just-in-Time) 컨텍스트는 다르다. 에이전트가 파일 경로나 쿼리 같은 식별자만 들고 있다가, 필요할 때 도구로 런타임에 적재한다. Claude Code의 head, tail, grep, glob, CLAUDE.md 하이브리드가 이 방식의 실전 구현이다.

긴 컨텍스트 LLM이면 RAG가 필요 없다는 이분법이 한때 유행했다. Li 등의 연구가 이것을 반증했다. 자원이 충분하면 긴 컨텍스트(LC)가 평균 성능에서 RAG를 앞서지만, RAG의 비용 우위는 그대로다. 대화형 질의와 일반 질의에서는 RAG가 여전히 유리하다. 또한 단순 청킹(naive chunking) RAG는 LC에 지지만, 요약 기반 검색을 적용하면 LC와 비등해진다. 진짜 변수는 **LC냐 RAG냐**가 아니라 **검색 전략 선택**인 셈이다. 실무 최적은 모델이 쿼리를 동적으로 라우팅하는 Self-Route 하이브리드다.

실전 사례: CLAUDE.md가 모든 것을 바꾼다

추상적 원칙만 나열하면 와닿지 않는다. Claude Code를 쓰는 사람이라면 CLAUDE.md를 알 것이다. 프로젝트 루트에 넣으면 매 세션 시작 시 자동으로 컨텍스트에 로드된다.

나쁜 CLAUDE.md는 이렇다. **이 프로젝트는 Next.js를 사용한다, 테스트를 먼저 실행하라, TypeScript를 사용한다.** 이것은 프롬프트다. **잘 해줘**와 다를 바 없다. 좋은 CLAUDE.md는 다르다. 파일 경로는 절대 경로만 사용할 것, 테스트는 단일 파일 먼저 돌리고 통과 후 전체를 돌릴 것, 커밋 훅 실패 시 원인을 수정하고 재시도할 것, 금지 사항(console.log 디버깅, any 타입)을 명시한다. 아키텍처 정보도 넣는다. API는 어디에, DB는 Supabase이고 마이그레이션은 어디에, 인증은 JWT와 refresh token rotation이라는 것까지.

차이는 **뭘 쓰는지**가 아니라 **어떻게 작업해야 하는지**를 알려주는 데 있다. Anthropic이 SWE-bench에서 발견한 사실이 있다. 상대 경로를 절대 경로로 바꾸는 것만으로 도구 사용 오류가 완전히 사라졌다. 도구 하나의 규칙이 전체 작업 품질을 좌우한 셈이다. CLAUDE.md는 Write 축의 구현이다. 규칙을 파일(윈도우 밖)에 저장하되, 매 세션 자동 로드로 항상 어텐션이 높은 위치에 배치하는 것이다.

실전: 100개 파일 리팩토링에 4축을 적용하면

추상적 원칙을 실전에 대입해보자. Next.js 앱에서 API 라우트 100개를 새 인증 시스템으로 마이그레이션해야 하는 상황이다. 프롬프트만으로 시도하면 AI가 파일 5개쯤 수정하다가 컨텍스트가 터진다. 100개 파일의 내용을 동시에 들고 있을 수 없기 때문이다.

축	적용	효과
Write	CLAUDE.md에 마이그레이션 규칙 저장. 기존 패턴과 새 패턴, 완료 기록 방식을 명시	매 세션마다 자동 로드. 100번 반복 설명 불필요
Select	파일 100개를 한번에 읽지 않고 grep으로 대상만 추출, 3 5개씩 로드	필요한 것만 로드해 윈도우 폭발 방지
Compress	파일 수정 완료 후 도구 출력을 한 줄 요약으로 대체	1,200줄 원본 대신 한 줄. 60 80% 절감
Isolate	메인 에이전트가 계획을 관리하고, 서브에이전트가 폴더별로 독립 작업	각자 깨끗한 컨텍스트에서 작업 후 결과만 반환

이렇게 하면 컨텍스트가 터지지 않고 100개 파일을 처리할 수 있다. 프롬프트를 잘 쓰는 것만으로는 절대 불가능한 작업이다. 컨텍스트 엔지니어링이란 **정보를 언제, 어떤 형태로, 얼마나, 어디에 배치할 것인가**를 설계하는 행위다.

70 80%의 법칙과 KV-Cache

컨텍스트 윈도우를 얼마나 채워야 하는가. 답은 70 80%까지다. 그 이상 채우면 성능이 무너지기 시작한다. Claude Code는 95%에서 자동 압축을 발동하지만 이것은 비상 브레이크다. 정상 운전에서는 70 80%를 넘기지 않아야 한다.

Claude Opus 4.6과 Sonnet 4.6은 1M 토큰 윈도우를 갖고 있다. 큰 윈도우니까 다 쓸 수 있을 것 같지만, Chroma 연구를 기억해야 한다. 200K 모델이 50K에서 이미 무너졌다. 1M 모델이라고 1M까지 안전한 것은 아니다.

이 제한이 존재하는 이유 중 하나가 KV-Cache 비용이다. 트랜스포머는 매 토큰 생성 시 이전 토큰들의 Key-Value 벡터를 참조한다. 이것이 KV-Cache다. 컨텍스트가 길수록 캐시가 커지고, 비용이 비선형으로 증가한다. 10K 토큰에서 100K 토큰으로 늘리면 비용이 10배가 아니라 그 이상 올라간다.

이 비용을 줄이는 핵심 기법이 프리픽스 캐싱(Prefix Caching)이다. 해시 기반 블록 매칭으로 동일한 프리픽스를 재사용하는 것이다. 시스템 프롬프트와 도구 정의처럼 매 요청마다 반복되는 부분을 캐싱하면 비용이 대폭 줄어든다. 이것이 컨텍스트 배치 순서에 영향을 준다. 안정적인 것을 앞에, 변하는 것을 뒤에 배치하면 캐시 히트율이 올라가고 비용이 내려간다. CLAUDE.md가 맨 앞에 로드되는 이유, 시스템 프롬프트가 항상 컨텍스트의 맨 앞에 있는 이유가 여기에 있다.

> 컨텍스트 윈도우는 70 80%까지만 채운다. 안정적인 요소를 앞에, 변하는 요소를 뒤에 배치해 프리픽스 캐싱 효율을 극대화한다. 비용과 성능을 함께 고려하면 **필요한 최소한**만 넣는 것이 항상 최선이다.

컨텍스트 엔지니어링은 건축이다

왜 **엔지니어링**이라는 이름이 붙었는지를 짚어야 한다. 프롬프트 엔지니어링에서의 **엔지니어링**은 솔직히 과장이었다. 문장을 잘 쓰는 것은 작문이지 엔지니어링이 아니다. 하지만 컨텍스트 엔지니어링은 진짜 엔지니어링이다.

Anthropic이 제시한 핵심 원리가 있다. 좋은 컨텍스트란 **원하는 결과의 가능성을 최대화하는 가장 작은 고신호 토큰 집합**이다. 이것은 정적 템플릿이 아니라 개발자, 사용자, 이전 상호작용, 도구 호출, 외부 데이터가 런타임에 조립되는 파이프라인이다. Harrison Chase의 정의를 빌리면, **LLM이 태스크를 그럴듯하게 해낼 수 있도록 올바른 정보와 도구를 올바른 형식으로 제공하는 동적 시스템을 구축하는 것**이다.

프롬프트를 쓰는 것이 페인트칠이라면, 컨텍스트를 설계하는 것은 건물의 골조를 세우는 것이다. 페인트칠도 중요하다. 하지만 골조가 부실하면 건물이 무너진다.

컨텍스트 엔지니어링의 한계, 그리고 다음

컨텍스트 엔지니어링은 프롬프트 엔지니어링의 한계를 넘어선 전환이다. 하지만 이것도 한계가 있다. 수동이라는 한계다.

Write, Select, Compress, Isolate 4축을 사람이 일일이 관리하는 것은 단발성 대화에서는 가능하다. **이 파일을 읽어서 이렇게 요약해줘**라고 말하는 것은 Select와 Compress를 수동으로 지시한 것이다. 그러나 100개 파일 리팩토링의 3시간짜리 세션에서, 일주일짜리 프로젝트에서 사람이 매 스텝마다 **지금 뭘 로드하고 뭘 버려**를 지시하는 것은 불가능하다.

이것을 자동화하는 소프트웨어가 필요하다. 컨텍스트를 모니터링하고, 예산을 관리하고, 필요 시 압축하고, 서브에이전트로 격리하는 소프트웨어 껍데기. 그것이 하네스(Harness)다. 다음 장에서 다룬다.

Sources: Karpathy 2025-06-25; Tobi Lutke 2025-06-19; Harrison Chase (LangChain) 2025-06-23; Simon Willison 2025-06-27; Lance Martin 2025-06-23; Chroma Research (Kelly Hong 외) 2025; Anthropic Platform Blog 2025-09; Cognition 2025; Li et al. (LC vs RAG) 2024; Schluntz & Zhang (Anthropic) 2024-12-19; KVTC 2025-11

CHAPTER 11

하네스: AI가 일하게 만드는 소프트웨어

앞 장에서 컨텍스트를 수동으로 관리하는 것의 한계를 확인했다. 자동화가 필요하다. 그 자동화를 담당하는 소프트웨어가 하네스다. 같은 모델이라도 하네스의 유무에 따라 채팅봇이 되기도 하고, 자율 에이전트가 되기도 한다.

세계에서 가장 똑똑한 모델이 가장 무능해지는 순간

Claude.ai 채팅 인터페이스에서 **이 프로젝트의 auth 모듈을 리팩토링해줘**라고 말해보자. 돌아오는 답은 **프로젝트의 코드를 보여주시면 도와드리겠습니다**다. 파일을 읽을 수 없고, 코드를 실행할 수 없고, 결과를 검증할 수 없다. 세계에서 가장 똑똑한 모델이 아무것도 못 하는 상태다.

같은 말을 Claude Code에서 해보자. Claude Code는 프로젝트 구조를 탐색하고(Glob, Grep), auth 관련 파일을 읽고(Read), 코드를 수정하고(Edit), 테스트를 실행하고(Bash), 실패하면 수정하고 다시 테스트한다. 같은 모델이다. Claude Opus 4.6. 달라진 것은 모델을 감싸는 소프트웨어, 즉 하네스다.

Boris Cherny(Anthropic, Claude Code 초기 개발자)가 이것을 **Product Overhang**이라 불렀다. **모델의 능력은 이미 충분한데, 그 능력을 제품으로 만드는 소프트웨어가 없었다.** Cherny가 처음 Claude에게 파일시스템 접근 권한을 줬을 때 **Claude가 파일시스템을 탐색하는 것이 경이로웠다**고 회고한 바 있다. 파일을 읽게 해줬더니 모든 것이 달라진 것이다.

> 하네스(Harness)란 LLM을 보이지 않는 프롬프트와 호출 가능한 도구로 확장하는 소프트웨어다(Simon Willison, 2026). 모델은 같은데 제품이 바뀐다. 채팅봇과 에이전트의 차이는 모델의 차이가 아니라 하네스의 차이다.

하네스는 두 가지를 한다

Simon Willison이 2026년 3월 16일에 정의한 것처럼, 하네스의 기능은 두 가지로 압축된다.

첫째, 보이지 않는 프롬프트를 관리한다. 사용자가 Claude Code에서 **이 파일 고쳐줘**라고 한 줄을 치면, 실제로 모델에게 전달되는 것은 그 한 마디가 아니다. 시스템 프롬프트 약 5,000 토큰, CLAUDE.md 내용, 도구 정의 목록, 이전 대화 기록이 앞에 붙는다. 사용자는 한 줄을 썼지만 모델은 수만 토큰을 받은 셈이다. 이 보이지 않는 부분을 관리하는 것이 하네스의 첫 번째 역할이다.

둘째, 호출 가능한 도구를 제공한다. 모델이 외부 세계와 상호작용하는 통로다. 모델이 **auth.ts 파일을 읽어야겠다**고 판단하면 Read 도구를 호출하고, 하네스가 실제로 파일을 읽어서 결과를 컨텍스트에 넣는다. 수정이 필요하면 Edit 도구를 호출하고, 테스트가 필요하면 Bash 도구를 호출한다. 모델이 **이 도구를 이 인자로 호출해줘**라고 요청하고, 하네스가 실제로 실행하고 결과를 돌려준다.

LLM은 무상태(stateless)라는 점이 핵심이다. Willison이 지적한 것처럼, **LLM은 매번 실행될 때마다 같은 백지에서 시작한다.** 대화가 이어지는 것처럼 보이지만 실제로는 하네스가 매번 전체 대화 이력을 다시 전송한다. 토큰 캐싱이 비용을 줄여주긴 하지만, 구조적으로 모든 턴마다 전체 컨텍스트가 재구성된다. 이 재구성을 자동으로 처리하는 것이 하네스의 일이다.

이 두 가지가 합쳐지면 채팅봇이 에이전트가 된다.

CLAUDE.md: 프로젝트의 두뇌를 설계하는 방법

보이지 않는 프롬프트 중 가장 중요한 것이 CLAUDE.md다. 프로젝트 루트에 놓는 파일 하나가 에이전트의 행동 전체를 바꾼다. CLAUDE.md는 하네스가 매 턴마다 자동으로 읽어서 시스템 프롬프트에 주입하는 프로젝트별 설정 파일이다.

나쁜 CLAUDE.md는 이렇게 생겼다.

나쁜 예: 추상적이고 실행 불가능

> 코드를 잘 짜주세요. 테스트도 꼼꼼히 하고 좋은 코드를 작성하세요. 사용자 경험을 중시합니다.

이것은 아무 정보도 주지 않는다. **좋은 코드**가 무엇인지, **테스트를 꼼꼼히**가 어떤 기준인지 모델은 알 수 없다. 주니어 개발자에게 이렇게 말하면 좋은 결과가 나오지 않는 것과 같다.

좋은 CLAUDE.md는 이렇게 생겼다.

좋은 예: 구체적이고 실행 가능

> **빌드/테스트 명령어**: npm run build, npm test, 단일 테스트는 npm test - -grep “test name”. **코드 스타일**: TypeScript strict mode, 함수형 우선, any 금지. 에러는 Result 타입으로 처리. **커밋 규칙**: conventional commits (feat:, fix:, refactor:). 커밋 전 반드시 npm test 통과 확인. **프로젝트 구조**: src/api/ (라우트), src/domain/ (비즈니스 로직), src/infra/ (DB, 외부 서비스). domain은 infra를 직접 import하지 않는다.

좋은 CLAUDE.md에는 네 가지가 들어간다. 빌드와 테스트 실행 방법, 코드 스타일과 규칙, 프로젝트의 디렉토리 구조와 의존성 방향, 그리고 커밋 규칙이다. 모델이 첫 턴에서 **이 프로젝트는 어떻게**

돌아가는가를 즉시 파악할 수 있어야 한다. CLAUDE.md가 잘 쓰여 있으면 모델이 프로젝트 탐색에 쓰는 턴 수가 줄어들고, 그만큼 실제 작업에 더 많은 컨텍스트 예산을 쓸 수 있다.

Simon Willison의 관련 개념인 AGENTS.md도 같은 원리다. 에이전트가 사용할 수 있는 패키지, 환경 정보, 예시 명령어를 문서화한 것이다. 핵심은 동일하다. 에이전트가 시행착오로 알아내야 할 것을 문서로 미리 알려주면 루프가 빨라진다.

도구 정의 설계: ACI가 프롬프트보다 중요하다

에이전트에게 도구를 주는 것만으로는 부족하다. 도구를 **어떻게 정의하느냐**가 결과를 결정한다.

Anthropic의 Erik Schluntz와 Barry Zhang(2024)이 SWE-bench를 풀면서 발견한 것이 있다. **프롬프트보다 도구 최적화에 더 많은 시간을 썼다.** 구체적으로, 파일 편집 도구에서 상대 경로를 절대 경로로 바꾸는 것만으로 도구 사용 오류가 완전히 사라졌다. 에이전트가 디렉토리를 이동한 후에 **auth.ts**를 수정하면 잘못된 위치의 파일을 수정하거나 파일을 아예 못 찾는다. /home/user/project/src/auth/auth.ts라고 절대 경로로 명시하면 이 문제가 사라진다. 사소해 보이는 변경이 전체 작업의 성패를 갈랐다.

Anthropic은 이 원칙을 ACI(Agent-Computer Interface) 디자인이라 부른다. 인간-컴퓨터 인터페이스(HCI)에 들이는 노력만큼을 에이전트-컴퓨터 인터페이스에도 들여야 한다는 것이다. 핵심 원칙은 세 가지다.

원칙	실천 방법
모델의 입장에서 생각할 것	모델이 도구를 처음 보는 주니어 개발자라고 상상하라. 암묵적 규칙은 없다
주니어 개발자에게 설명하듯 문서를 쓸 것	파라미터 이름, 반환값 형식, 에러 시 행동을 명시하라. 도구 설명은 수백 줄이 될 수 있다
실수가 어렵도록 설계할 것(포카요케)	상대 경로 대신 절대 경로를 강제하라. 위험한 기본값을 제거하라

도구 정의 자체가 컨텍스트 윈도우의 예산을 잡아먹는다는 점도 중요하다. Claude Code의 시스템 프롬프트에는 도구 정의만으로 수천 토큰이 들어간다. 도구가 많을수록 컨텍스트가 줄어든다. 도구는 적게, 하지만 정확하게 설계하는 것이 원칙이다.

Claude 4에서 관찰된 현상이 이 원칙을 뒷받침한다. 멀티에이전트 시스템에서 Claude 4가 자기 도구 설명을 자율적으로 재작성한 결과, 태스크 완료 시간이 40% 감소했다. 도구 설명이 곧 성능이라는 증거다. 나쁜 도구 설명은 나쁜 프롬프트보다 더 치명적이다. 프롬프트는 한 번 읽히지만, 도구 정의는 매 턴마다 컨텍스트에 포함되기 때문이다.

에이전트 루프: 수집, 행동, 검증, 반복

하네스의 심장은 루프다. 이 루프의 원형은 Yao 등(2022)의 ReAct 패턴이다. **추론 흔적(reasoning trace)**과 **행동(action)**을 교차시키는 것이다. 모델이 **무엇을 해야 하는지** 추론하고, 그에 따라 행동하고, 결과를 관찰하고, 다시 추론한다. 이 패턴이 현대 에이전트 루프의 기반이 됐다.

Anthropic의 Thariq Shihipar(2025)가 이 원형을 실전에 맞게 재정리한 패턴이 표준이 됐다. **Gather context, Take action, Verify work, Repeat.**

그림 3: 워크플로우와 에이전트의 차이. 워크플로우는 경로가 사전에 고정돼 있고, 에이전트는 모델이 상황에 따라 다음 행동을 스스로 결정한다.

실전 시나리오로 이 루프를 따라가보자. **로그인이 안 된다는 버그 리포트**가 들어왔다.

단계	행동	내용
Gather	grep, Read	src/ 에서 login 키워드 검색. 관련 파일 5개 발견. login.ts 코드 확인
Think	추론	42번 줄에서 토큰 만료 체크가 빠져 있음을 발견
Act	Edit	login.ts 42번 줄 수정
Verify	Bash	npm test auth.test.ts 실행. 1건 실패
Think	추론	타임존 문제 파악. UTC 변환 필요
Act	Edit	login.ts 재수정
Verify	Bash	auth 테스트 통과. 전체 테스트 통과

이 전체 과정에서 사람이 개입한 것은 처음 **로그인 버그 고쳐줘** 한 마디뿐이다. 나머지는 하네스의 에이전트 루프가 자동으로 돌린 것이다. 수집(grep, Read), 행동(Edit), 검증(Bash: test), 실패 시 다시 수집부터.

검증 단계가 빠지면 치명적이다. 검증 없는 에이전트는 **고쳤다**고 보고하면서 실제로는 망가뜨린 상태가 된다. Anthropic이 제시한 검증 3단계가 있다.

단계	방법	특성
1단계	규칙 기반 (lint, type check, test)	가장 빠르고 저렴. 자동화 용이
2단계	시각적 피드백 (브라우저 스크린샷)	사람이 보는 것과 같은 확인. 중간 비용

3단계	LLM-as-Judge	가장 비싸지만 유연. 복잡한 판단 가능

장기 작업에서는 단위 테스트보다 브라우저 기반 E2E 검증이 효과적이다. Justin Young(Anthropic, 2025)의 실험에서 Puppeteer MCP로 브라우저 자동화 도구를 제공하자 에이전트의 작업 품질이 크게 향상됐다. Playwright 스크린샷으로 실제 화면을 캡처해서 모델에게 **이게 맞는 결과인가**를 판단시키는 방식이다. 비용이 들지만, 인간이 보는 것과 가장 가까운 검증이다.

6.7B가 175B를 이기는 이유: 도구의 힘

도구 사용이 왜 중요한지를 보여주는 결정적 연구가 있다. Schick 등(Meta AI, 2023)의 Toolformer다. 자기지도학습으로 도구 사용법을 배운 모델이 후보 도구 호출을 생성하고 실행한 뒤, 미래 토큰 퍼플렉시티가 낮아지는 것만 보존하는 방식으로 학습했다. 결과는 놀라웠다. 6.7B 파라미터에 도구를 결합해 GPT-3 175B 수준의 성능을 달성했다. 수학, 질의응답, 시간 추론에서 모델 크기의 25배 차이를 도구 사용이 메운 셈이다.

비유하면 이렇다. 암산만으로 복잡한 수학 문제를 풀려는 천재와, 계산기를 든 보통 사람이 경쟁하는 상황이다. 계산기가 있으면 보통 사람도 천재를 이길 수 있다. LLM에게 도구는 계산기와 같다. 모델의 내재 능력을 키우는 것보다 적절한 도구를 주는 것이 때로 더 효과적이다.

하네스 설정 과정: 새 프로젝트에서 시작하기

새 프로젝트에서 하네스를 세팅하는 과정은 네 단계로 나뉜다. 이 과정을 제대로 밟으면 에이전트가 처음부터 맥락을 가진 채 작업을 시작한다. 대충 밟으면 에이전트가 매번 프로젝트를 탐색하는 데 컨텍스트의 절반을 낭비한다.

1단계: CLAUDE.md 작성. 프로젝트 루트에 CLAUDE.md를 만든다. 빌드 명령어, 테스트 명령어, 코드 스타일 규칙, 디렉토리 구조를 적는다. 이것이 에이전트의 첫 번째 지식이 된다. 기존 프로젝트라면 README에서 핵심 정보를 추출해 에이전트용으로 재구성하면 된다. 핵심은 **사람이 읽는 문서**가 아니라 **에이전트가 실행하는 지침**이라는 점이다.

2단계: 도구 범위 결정. 에이전트가 사용할 도구의 범위를 정한다. 파일 읽기와 쓰기는 기본이다. 셸 명령 실행을 허용할 것인가, 브라우저 자동화를 붙일 것인가, 외부 API 호출을 허용할 것인가를 결정한다. Simon Willison의 조언대로, MCP에 의존하기보다 셸 명령으로 해결할 수 있는 것은 셸 명령으로 하는 편이 투명하고 디버깅이 쉽다.

3단계: 권한 경계 설정. 읽기 작업은 자동 허용, 쓰기 작업은 상황에 따라, 파괴적 작업은 반드시 승인으로 나눈다. YOLO 모드를 쓴다면 반드시 샌드박스 환경이나 스테이징 크리덴셜과 짝지어야 한다. Willison은 Fly.io 실험에서 에이전트 전용 조직을 5달러 예산 한도로 만들어 격리한 사례를 소개했다.

4단계: 검증 루프 구축. 테스트 명령어가 있으면 에이전트가 매 수정 후 자동으로 검증할 수 있다. 테스트가 없는 프로젝트라면 lint와 type check라도 설정한다. 검증 도구가 없으면 에이전트 루프의 핵심이 빠진 것이다. 자동화된 테스트가 에이전트의 가치를 극적으로 증폭한다는 것이 Willison의 핵심 관찰이다.

이 네 단계를 한 마디로 요약하면 이렇다. 에이전트가 **무엇을 알아야 하는지**(CLAUDE.md), **무엇을 할 수 있는지**(도구), **무엇을 하면 안 되는지**(권한), **어떻게 확인하는지**(검증)를 미리 정해주는 것이다.

Before/After: 하네스의 유무가 만드는 차이

같은 모델, 같은 작업을 하네스 없이 했을 때와 있을 때를 비교하면 차이가 극명하다.

작업	하네스 없이 (채팅)	하네스 있게 (에이전트)
버그 수정	사용자가 코드를 복사해 붙여넣고, 모델의 수정안을 다시 복사해 파일에 붙여넣고, 직접 테스트를 돌린다. 실패하면 에러 로그를 다시 복사해 대화에 붙여넣는다	사용자가 **로그인 버그 고쳐줘** 한 마디. 에이전트가 파일 탐색, 수정, 테스트, 재수정을 자동으로 반복한다
리팩토링	파일마다 하나씩 보여주고, 수정안을 받아 적용하고, 다음 파일을 보여준다. 10개 파일이면 20번의 복사-붙여넣기	에이전트가 프로젝트 전체를 탐색해 관련 파일을 찾고, 의존성을 분석해 순서대로 수정하고, 전체 테스트를 돌려 확인한다
새 기능 개발	사용자가 요구사항을 설명하고, 모델이 코드를 생성하면 사용자가 파일에 넣고 실행해본다. 오류가 나면 다시 대화	사용자가 요구사항을 설명하면 에이전트가 파일 생성, 라우트 추가, 테스트 작성, 실행, 수정까지 자동으로 진행한다

차이의 본질은 **누가 루프를 도는가**다. 하네스 없이는 사람이 수집-행동-검증 루프의 모든 단계를 직접 실행한다. 하네스가 있으면 소프트웨어가 루프를 돌고 사람은 시작 명령만 내린다. 같은 버그 수정에 채팅에서는 30분이 걸리고 에이전트에서는 3분이 걸리는 이유가 여기에 있다. 모델의 추론 속도가 빨라진 것이 아니다. 사람이 병목이던 구간이 자동화된 것이다.

장기 실행 에이전트: 하루짜리 작업을 맡기려면

5분짜리 버그 수정이 아니라 3시간짜리 기능 개발이나 하루짜리 리팩토링을 에이전트에게 맡기려면 한 단계 더 나가야 한다. LLM은 무상태(stateless)다. Justin Young(Anthropic, 2025)의 표현대로, **새 세션은 이전에 무엇을 했는지 아무 기억 없이 시작된다.** 하지만 git commit과 progress 파일은 유상태(stateful)다. 세션이 끊겨도 살아 있다.

Young이 이 문제를 2부 구조로 해결했다.

초기화 에이전트(Initializer). 프로젝트를 처음 받으면 init.sh 스크립트를 실행하고, claude-progress.txt 파일을 생성하고, 200개 이상의 기능 요구사항을 JSON으로 정리하고, 초기 git commit을 만든다. 모든 기능은 처음에 **실패** 상태로 표시된다. 이것이 에이전트에게 명확한 작업 목록과 진척 측정 기준을 제공한다.

코딩 에이전트(Coder). 매 세션 시작 시 다음 순서를 따른다. 현재 디렉토리를 확인하고, git log를 리뷰해 이전 세션에서 무엇을 했는지 파악하고, claude-progress.txt를 읽어 미완료 기능을 찾고, 개발 서버를 띄운 뒤 기존 기능이 깨지지 않았는지 먼저 확인하고, 그 다음에 새 기능 하나를 구현한다.

이 구조의 핵심은 **파일시스템이 메모리**라는 점이다. LLM의 컨텍스트 윈도우는 세션이 끝나면 사라지지만, git history와 progress 파일은 영속한다. 하네스가 무상태 LLM의 한계를 유상태 외부 시스템으로 극복하는 셈이다.

장기 실행에서 발견된 네 가지 실패 모드와 대응이 있다.

실패 모드	증상	대응
너무 많은 동시 작업	구현 중 컨텍스트 소진. 절반만 완성된 기능이 여러 개	JSON 기능 목록으로 단일 기능 단위 작업. 한 번에 하나만
조기 완료 선언	절반도 안 됐는데 **완료**라고 보고	브라우저 자동화로 자가 검증. 화면에서 직접 확인
문서 없는 코드 방치	나중에 아무도 이해 불가	구현 완료 시 README와 progress 파일 갱신 의무화
앱 실행 방법 미숙지	에이전트가 서버를 못 띄움	init 스크립트에 시작 절차 명시. CLAUDE.md에 실행 명령어 기록

두 번째 실패 모드가 특히 치명적이다. 에이전트가 테스트를 지우거나 수정해서 **통과**시키는 경우, 아무것도 검증되지 않은 것이다. Young의 명시적 경고가 있다. **에이전트가 테스트를 지우거나 수정하게 허용하는 것은 용납할 수 없다.** 검증 루프의 기반 자체가 사라지기 때문이다.

서브에이전트: 컨텍스트를 분리하는 기술

하나의 에이전트가 모든 것을 하면 컨텍스트가 빠르게 소진된다. 10개 파일을 읽고, 3개를 수정하고, 테스트를 돌리면 컨텍스트 윈도우의 상당 부분이 이미 찬 상태다. 이때 추가 작업을 맡기면 앞서 읽은 파일의 내용이 컴팩션(요약)되면서 세부 정보가 사라진다.

서브에이전트는 이 문제의 해법이다. 메인 에이전트가 특정 작업을 격리된 컨텍스트 창에서 실행되는 서브에이전트에게 위임하는 패턴이다. Claude Code에서 이 기능은 3일 만에 구축됐고, 개발 과

정에서 2일치 작업이 폐기됐다(Boris Cherny, 2025). 빠르게 시도하고 빠르게 버리는 개발 방식의 전형이다.

서브에이전트의 원리는 이렇다. 메인 에이전트가 **이 파일의 타입 에러를 모두 수정해**라는 작업을 서브에이전트에게 넘긴다. 서브에이전트는 자기만의 컨텍스트 윈도우에서 해당 파일을 읽고, 수정하고, 검증한다. 완료되면 결과 요약만 메인 에이전트에게 돌려준다. 메인 에이전트의 컨텍스트에는 서브에이전트가 읽은 파일 전체가 아니라 **타입 에러 5건 수정 완료**라는 요약만 들어간다.

이 패턴이 유효한 경우와 그렇지 않은 경우가 있다.

서브에이전트가 유효한 경우	서브에이전트가 불필요한 경우
독립적인 파일이나 모듈 단위 작업	파일 간 의존성이 높아 전체 맥락이 필요한 작업
탐색적 작업 (여러 디렉토리에서 정보 수집)	한 파일 안에서의 단순 수정
병렬 처리가 가능한 반복 작업	사용자와의 대화 맥락이 중요한 작업

Anthropic의 Building Effective Agents 문서에서 제시한 **Orchestrator-Workers** 패턴이 이 구조의 일반형이다. 중앙 LLM이 작업을 동적으로 분해하고, 워커 LLM에게 위임하고, 결과를 합성한다. 서브에이전트는 이 패턴의 코딩 에이전트 구현이다.

장기 실행 에이전트에서 서브에이전트의 가치는 더 커진다. 메인 에이전트가 3시간 동안 작업하면서 컨텍스트가 누적되면, 새 작업을 깨끗한 컨텍스트에서 시작하는 서브에이전트가 오히려 더 정확하다. 오래된 컨텍스트에 묻혀 있는 메인 에이전트보다 방금 시작한 서브에이전트가 단일 작업에서는 더 나은 결과를 내는 역설이 발생한다.

Claude Code의 숫자들: 하네스가 만드는 차이

Claude Code가 하네스의 대표 구현이라고 했다. 숫자로 보면 이 하네스가 얼마나 강력한지 실감할 수 있다. The Pragmatic Engineer의 Gergely Orosz가 Boris Cherny, Sid Bidasaria(창립 엔지니어), Cat Wu(PM)를 인터뷰한 기사에서 공개된 수치다.

지표	수치
연간 매출 런레이트	5억 달러 이상 (2025년 5월 일반 출시 이후)
성장률	3개월 만에 사용량 10배
내부 도입 1일차	Anthropic 엔지니어의 20%
내부 도입 5일차	50%
자체 코드 비율	Claude Code의 약 90%가 Claude Code로 작성
릴리즈 속도	내부 60 100회/일, 외부 1회/일
엔지니어당 PR	하루 약 5개

설계 철학에서 나온 핵심 원칙이 있다. **비즈니스 로직을 최소화하고, 모델이 대부분의 일을 한다. 새 모델이 나올 때마다 코드를 지운다.** 이것이 의미하는 바는 명확하다. 좋은 하네스는 모델을 바꿔도 제품이 유지된다. 나쁜 하네스는 모델을 바꾸면 전체를 다시 써야 한다.

기술 스택도 이 철학을 반영한다. TypeScript, React with Ink(터미널 UI), Bun(빌드). 이 조합을 선택한 이유가 있다. Claude가 이미 인터넷에서 가장 많이 본 기술, 즉 **분포 위에 있는(on distribution)** 기술이기 때문이다. 모델이 잘 아는 언어로 하네스를 만들면 모델이 자기 하네스를 개선할 수 있다. Claude Code의 90%가 Claude Code 자체를 이용해 작성됐다는 사실이 이 원칙의 증거다.

반패턴: 이렇게 하면 망한다

세 가지 대표적 반패턴이 있다.

첫째, 워크플로우를 에이전트라고 부르는 것이다. 입력을 받아 요약하고, 번역하고, 포맷팅하는 세 단계가 if-else로 고정돼 있으면 이것은 워크플로우다. Anthropic의 정의가 명확하다. **워크플로우는 LLM과 도구가 사전 정의된 코드 경로로 오케스트레이션되는 시스템이고, 에이전트는 LLM이 자신의 프로세스와 도구 사용을 동적으로 지휘하는 시스템이다.** 워크플로우는 경로가 코드에 박혀 있고, 에이전트는 경로가 모델의 추론에 의해 런타임에 결정된다. 워크플로우가 나쁜 것이 아니다. 잘 정의된 작업에는 워크플로우가 예측 가능하고 일관성 있다. 문제는 워크플로우를 에이전트라고 부르면서 에이전트의 유연성을 기대하는 것이다.

둘째, 비즈니스 로직을 하네스에 직접 넣는 것이다. 파이썬 파일이면 Black을 돌리고 타입스크립트 파일이면 Prettier를 돌리는 분기를 하네스에 넣으면 새 언어가 추가될 때마다 코드를 수정해야 한다. 이런 규칙은 CLAUDE.md나 Skills에 넣어야 한다. 하네스는 인프라여야 한다.

셋째, 테스트를 에이전트가 수정하거나 삭제하게 허용하는 것이다. AI가 **테스트를 수정해서 통과시켰습니다**라고 하면 아무것도 검증되지 않은 것이다. 검증 루프의 기반이 무너지기 때문이다.

> 좋은 하네스의 리트머스 테스트는 세 가지다. 모델을 교체해도 제품이 유지되는가. 비즈니스 로직이 하네스가 아니라 설정에 있는가. 에이전트가 테스트를 수정하지 못하게 막고 있는가.

에이전틱 루프 설계: 새로운 핵심 스킬

Simon Willison은 **에이전틱 루프를 설계하는 능력**이 2025년에 새로 이름 붙은 스킬이라고 말했다. Claude Code가 2025년 2월에 출시된 이후 불과 1년 만에 정의된 직업적 역량이다. 이 스킬의 핵심은 도구와 루프를 어떻게 조합하느냐에 있다.

효과적인 에이전틱 루프의 조건은 명확하다. 성공 기준이 분명해야 한다. 시행착오가 수반될 가능성이 높아야 한다. 자동화된 검증이 가능해야 한다. Willison이 열거한 이상적 후보 작업이 있다.

작업	왜 에이전틱 루프에 적합한가
실패하는 테스트 디버깅	성공 기준(테스트 통과)이 자동으로 측정된다
느린 SQL 쿼리 최적화	실행 시간이 수치로 나온다. 개선 여부가 즉시 확인된다
의존성 업그레이드	빌드 성공/실패가 자동 판정된다
컨테이너 크기 축소	이미지 크기가 숫자로 나온다. 기능 유지는 테스트로 확인한다

공통점이 있다. 전부 **기계가 판정할 수 있는 성공 기준**이 있다. 반대로, **이 디자인이 예쁜가**나 **이 문장이 자연스러운가**는 에이전틱 루프에 적합하지 않다. 기계가 판정할 수 없으면 루프가 돌 수 없다.

프레임워크 선택에 대해 Anthropic은 명확한 입장을 갖고 있다. LLM API 직접 호출로 시작하라는 것이다. Claude Agent SDK, AWS Strands Agents SDK 같은 프레임워크가 편의를 주지만, 프레임워크가 프롬프트와 응답을 가리면 디버깅이 무너진다. 추상화 계층이 편의를 주지만, 문제가 생겼을 때 어디서 잘못됐는지 알 수 없게 만든다. 간단한 에이전트 루프는 몇 줄의 코드로 구현할 수 있다. Willison의 표현대로 **LLM** + **시스템 프롬프트** + **도구를 루프에 넣으면** 기본 구현은 끝이다.

하네스 경쟁: Claude Code만 있는 것이 아니다

2026년 현재 주요 하네스들이 경쟁하고 있다.

하네스	접근 방식	특징
Claude Code	최소 비즈니스 로직, 자율 에이전트	Anthropic 직접 개발. 모델이 대부분 결정
Cursor	IDE 내장형	코드 인덱싱 + AI 자동완성이 핵심
GitHub Copilot	기존 워크플로우에 AI 삽입	VS Code/JetBrains 통합
Windsurf (Codeium)	IDE 자체가 하네스	플로우 기반 작업 관리
Aider	오픈소스 CLI	가볍고 다중 모델 지원

공통점이 있다. 전부 **LLM** + **파일시스템** + **도구 호출** + **검증 루프**라는 같은 뼈대를 공유한다. 차이는 어디에 중점을 두느냐다. IDE 통합(Cursor, Copilot)이냐, 자율 에이전트(Claude Code)냐.

권한과 안전: 돌이킬 수 없는 동작을 어떻게 다루는가

에이전트에게 도구를 주면 능력이 생기지만, 위험도 함께 생긴다. 파일을 수정할 수 있다는 것은 파일을 잘못 수정할 수도 있다는 뜻이다. 코드를 실행할 수 있다는 것은 위험한 명령어를 실행할 수도 있다는 뜻이다.

Simon Willison이 코딩 에이전트의 위험을 세 가지로 정리했다. 첫째, 나쁜 셸 명령이 중요한 파일을 삭제하거나 손상시키는 것. 둘째, 유출 공격(exfiltration attack)으로 에이전트가 볼 수 있는 파일이나 데이터를 외부로 빼내는 것. 셋째, 에이전트의 머신을 DDoS 프록시나 해킹 도구로 악용하는 것이다.

균형점은 이렇다. 읽기 작업(grep, Read)은 자동 허용한다. 쓰기 작업(Edit)은 상황에 따라 자동 또는 승인한다. 파괴적 작업(삭제, push, 프로덕션 접근)은 반드시 승인한다. YOLO 모드(전부 자동 허용)를 쓸 때는 샌드박스 환경, 스테이징 크리덴셜, 예산 한도와 짝지어야 한다.

위험 완화 옵션을 Willison은 세 가지로 나눴다. 샌드박스 환경을 사용하는 것, 남의 컴퓨터(GitHub Codespaces 같은)를 사용하는 것, 또는 위험을 감수하되 악성 지시 소스를 피하는 것이다. 이 균형을 잘못 잡으면 두 가지 극단에 빠진다. 하나는 모든 걸 허용해서 사고가 나는 것이고, 다른 하나는 모든 걸 막아서 에이전트가 무력해지는 것이다. 매 명령에 승인을 요구하면 에이전트가 시행착오로 문제를 풀어가는 능력, 즉 에이전트의 핵심 가치가 사라진다.

하네스의 다음: 외부 세계와의 연결

하네스가 AI를 **일하게** 만들었다. 파일을 읽고, 코드를 수정하고, 테스트를 돌리는 것은 이제 된다. 하지만 외부 시스템과의 연결은 다른 문제다. GitHub에서 이슈를 가져오고, Slack에서 메시지를 보내고, DB에서 쿼리를 실행하려면 하네스에 외부 세계의 팔을 달아주는 표준이 필요하다.

그 표준이 MCP(Model Context Protocol)다. 하지만 이 프로토콜에는 약속만큼이나 문제도 있었다. PART IV에서 다룬다.

Sources: Boris Cherny via Gergely Orosz, The Pragmatic Engineer 2025-09-23; Simon Willison 2026-03-16; Simon Willison, Designing agentic loops 2025-09-30; Erik Schluntz & Barry Zhang, Anthropic 2024-12-19; Thariq Shihipar, Anthropic 2025-09-29; Justin Young, Anthropic 2025-11-26; Schick et al. (Toolformer, Meta AI) 2023; Yao et al. (ReAct) 2022